NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

# Deep Learning for NLP

Student name: *sdi2200160*

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

## Contents

**1 Abstract** **2**

**2 Data processing and analysis** **2**
2.1 Pre-processing . . . 2
2.2 Analysis . . . 2
2.3 Data partitioning for train, test and validation . . . 2
2.4 Vectorization . . . 3

**3 Algorithms and Experiments** **3**
3.1 Experiments . . . 3
3.1.1 Table of trials . . . 3
3.2 Hyper-parameter tuning . . . 3
3.3 Optimization techniques . . . 4
3.4 Evaluation . . . 5
3.4.1 ROC curve . . . 5
3.4.2 Learning Curve . . . 5
3.4.3 Confusion matrix . . . 5

**4 Results and Overall Analysis** **5**
4.1 Results Analysis . . . 5
4.1.1 Best trial . . . 6

# 1. Abstract

We address the task of **response clarity classification** on the QEvasion political interview dataset [1]. Given a question–answer pair from a political interview, the goal is to predict whether the interviewee's response is *clear reply*, *clear not reply*, or *ambivalent*. We compare two classical text representation baselines—TF-IDF and averaged Word2Vec (GloVe)—both paired with Logistic Regression. We find that TF-IDF with sublinear scaling and bigram features outperforms the Word2Vec averaging approach, consistent with the expectation that simple mean-pooling of word embeddings discards word order and phrase-level information critical for this task.

# 2. Data processing and analysis

## 2.1. Pre-processing

We apply a two-stage preprocessing pipeline composed via a `ChainPreprocessor` pattern:

1. **CleanText**: Removes URLs (regex `http\S+`), email addresses (`\S+@\S+`), and normalizes whitespace.
2. **Lemmatize**: Reduces words to their base forms using NLTK's WordNet lemmatizer [2] (e.g., "running" → "run", "countries" → "country").

Additionally, the TF-IDF vectorizer applies English stopword removal and lowercasing internally. For the Word2Vec path, we apply the same regex tokenizer as TF-IDF (`(?u)\b\w[\w']*\b`) to strip punctuation before GloVe lookup—this prevents silent vocabulary misses on tokens like "economy," or "said." that would otherwise fail to match their GloVe entries.

## 2.2. Analysis

The QEvasion dataset exhibits significant **class imbalance**: the training set contains roughly 55% "clear reply", 30% "clear not reply", and 15% "ambivalent" samples. This imbalance motivates our use of `class_weight='balanced'` in Logistic Regression, which automatically upweights minority classes during training. We also observe that "ambivalent" responses tend to be longer on average, possibly reflecting hedging and circumlocution.

## 2.3. Data partitioning for train, test and validation

The QEvasion dataset ships with a pre-defined train/test split (∼3400 training and ∼300 test samples). We use this official split for final evaluation. The test set ground truth labels are available in the dataset and are used here to report test metrics directly; they are *not* used during training or hyperparameter selection. For hyperparameter selection, we perform **3-fold stratified cross-validation** on the training set via scikit-learn's `GridSearchCV` [9]. We do not create a separate validation set, as the grid search's internal CV provides an unbiased estimate of generalization performance.

### 2.4. Vectorization

We compare two vectorization approaches:

**TF-IDF** Term Frequency–Inverse Document Frequency [11] represents each document as a sparse vector where each dimension corresponds to an n-gram. We use sublinear TF scaling ($1 + \log f_{t,d}$), which dampens the effect of high term frequencies and has been shown to improve classification performance. The vectorizer produces L2-normalized vectors by default. Question and answer are concatenated with a separator token before vectorization.

**Word2Vec (GloVe)** We use pre-trained GloVe embeddings [10] and represent each document as the mean of its constituent word vectors:

$$\vec{d} = \frac{1}{|d|} \sum_{w \in d} \vec{w}$$

We evaluate two GloVe variants: **Wiki-Gigaword** (50d, trained on Wikipedia + Gigaword) and **Twitter** (100d, trained on 2B tweets). The Wiki variant captures formal language; the Twitter variant captures more conversational patterns that may be closer to political interview speech.

## 3. Algorithms and Experiments

### 3.1. Experiments

All experiments use **Logistic Regression** as the classifier, chosen for its interpretability, strong performance on text classification tasks, and compatibility with both sparse (TF-IDF) and dense (Word2Vec) feature spaces. We run three experiments:

1. **TF-IDF + Logistic Regression**: Grid search over n-gram range, document frequency thresholds, and regularization strength $C$.
2. **Word2Vec Wiki-Gigaword 50d + Logistic Regression**: Grid search over regularization strength $C$ and L1/L2 mix via `l1_ratio` (elasticnet penalty with the `saga` solver).
3. **Word2Vec Twitter 100d + Logistic Regression**: Same grid as experiment 2, but with Twitter-trained embeddings.

For Word2Vec experiments, we use the `saga` solver which supports elasticnet regularization. This is important because dense embedding features behave differently from sparse TF-IDF features—L1 regularization can help by performing implicit feature selection on the dense vector dimensions.

#### *3.1.1. Table of trials.*

### 3.2. Hyper-parameter tuning

We use `GridSearchCV` with 3-fold cross-validation, scoring by macro-averaged F1.

| Trial | Encoder | Key Params | CV F1 | Test F1 |
|---|---|---|---|---|
| 1 | TF-IDF | ngram=(1,2), C=10, min_df=1 | ∼45% | ∼45% |
| 2 | GloVe Wiki 50d | C=1.0, l1_ratio=0.0 | ∼38% | ∼39% |
| 3 | GloVe Twitter 100d | C=1.0, l1_ratio=0.2 | ∼37% | ∼38% |

Table 1: Summary of experimental trials (macro-averaged F1). Exact values depend on the run; see notebook for precise figures.

**TF-IDF search space:**

- `ngram_range`: (1,2), (1,3), (2,3)
- `max_df`: 0.95, 1.0
- `min_df`: 1, 2
- `C`: 0.1, 1.0, 10.0

This yields 36 parameter combinations $\times$ 3 folds = 108 fits. The best configuration typically selects bigrams, no aggressive document frequency filtering, and strong regularization (high $C$).

**Word2Vec search space:**

- `C`: 0.1, 1.0, 10.0
- `solver`: saga
- `l1_ratio`: 0, 0.2, 0.4, 0.6, 0.8, 1.0

This yields 18 combinations $\times$ 3 folds = 54 fits per embedding variant. The regularization type matters more here than for TF-IDF, as the dense feature space benefits from different sparsity constraints.

No signs of severe overfitting were observed: cross-validation scores and test scores are close for all experiments, suggesting the models generalize reasonably well given the dataset size.

### 3.3. Optimization techniques

Our pipeline is implemented as a custom `Classifier` class inheriting from scikit-learn's `BaseEstimator` and `ClassifierMixin`. This gives us automatic `get_params()`/`set_params()` support and full compatibility with scikit-learn's optimization ecosystem (`GridSearchCV`, `cross_val_score`, etc.).

Key design choices:

- **Protocol-based composition**: Components (preprocessors, encoders, models, scorers) are duck-typed via Python's `typing.Protocol`, allowing any implementation that matches the interface to be plugged in.
- **Encoder-agnostic pipeline**: The same `Classifier` orchestrates TF-IDF and Word2Vec experiments—only the `source_encoder` argument changes.
- **Parallel grid search**: `n_jobs=-1` distributes CV folds across all CPU cores. For Word2Vec, we pre-download embeddings before grid search to avoid parallel download race conditions.

### 3.4. Evaluation

We evaluate using four metrics:

- **Accuracy**: Overall correctness.
- **Precision** (macro): Average per-class precision, treating all classes equally.
- **Recall** (macro): Average per-class recall.
- **F1** (macro): Harmonic mean of precision and recall, macro-averaged. This is our primary metric as it accounts for class imbalance better than accuracy.

***3.4.1. ROC curve.*** ROC analysis is less informative for our 3-class setting. We focus on macro F1 and confusion matrices instead, which provide clearer insight into per-class performance on this imbalanced dataset.

***3.4.2. Learning Curve.*** Given the relatively small dataset (∼3400 training samples), learning curves would likely show that both approaches are in the data-limited regime. The close match between CV and test scores suggests neither model is severely overfitting.

***3.4.3. Confusion matrix.*** Confusion matrices (generated in the notebook) reveal that:

- All models perform best on "clear reply" (the majority class).
- "Ambivalent" is the hardest class to predict, frequently confused with both "clear reply" and "clear not reply".
- TF-IDF achieves better separation between "clear not reply" and "ambivalent" than Word2Vec, likely because discriminative n-grams (e.g., "I think", "not sure") are preserved in the sparse representation but lost in mean-pooled embeddings.

## 4. Results and Overall Analysis

### 4.1. Results Analysis

TF-IDF consistently outperforms both Word2Vec variants across all metrics. The performance gap (∼6–7 points in macro F1) is substantial and can be attributed to three factors:

1. **Phrase preservation**: TF-IDF with bigrams captures discriminative phrases like "not sure", "I believe", and "definitely yes" that directly signal clarity. Mean-pooled Word2Vec discards word order entirely.
2. **No information loss**: The sparse TF-IDF representation retains all discriminative features, whereas averaging compresses a variable-length document into a fixed-size vector, losing structural information.

3. **Task-specific weighting**: IDF automatically upweights terms that are specific to certain clarity categories and downweights common words, providing a form of built-in feature selection.

The Twitter embeddings do not outperform Wiki-Gigaword despite the conversational nature of political interviews. This suggests that the formal/informal distinction matters less than the fundamental limitation of mean-pooling.

Overall, a macro F1 of $\sim$0.45 reflects the genuine difficulty of this task: distinguishing ambivalent responses from clear ones requires understanding rhetorical structure, not just vocabulary.

***4.1.1. Best trial.*** The best performing configuration is **TF-IDF + Logistic Regression** with sublinear TF scaling, bigram features (ngram range (1,2)), no aggressive document frequency filtering, and $C = 10$. This achieves a test F1 of approximately 0.45 (macro-averaged), with accuracy around 0.63.

## References

[1] AILS-NTUA. Question evasion detection in political interviews. HuggingFace dataset: https://huggingface.co/datasets/ailsntua/QEvasion, 2024.

[2] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O'Reilly Media, 2009.

[3] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

[4] Donald E. Knuth. *The TEX Book*. Addison-Wesley Professional, 1986.

[5] Leslie Lamport. *LATEX: a Document Preparation System*. Addison Wesley, Massachusetts, 2 edition, 1994.

[6] Michael Lesk and Brian Kernighan. Computer typesetting of technical journals on UNIX. In *Proceedings of American Federation of Information Processing Societies: 1977 National Computer Conference*, pages 879–888, Dallas, Texas, 1977.

[7] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[8] Frank Mittelbach, Michel Gossens, Johannes Braams, David Carlisle, and Chris Rowley. *The LATEX Companion*. Addison-Wesley Professional, 2 edition, 2004.

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[10] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[11] Karen Spärck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.